

Module 4: Planning & Probabilistic Reasoning

STRIPS, PDDL, Graphplan Mutex, Bayes' Rule, Bayesian Belief Networks & d-separation

Module IV: Planning & Probabilistic Reasoning in AI

- | | |
|---|---|
| 1. Classical Planning Formalisms (State, Goal & Action Representations) | 2. The STRIPS Language: Preconditions, Add Lists & Delete Lists |
| 3. Planning Domain Definition Language (PDDL) Action Schemas | 4. Progression (Forward State Search) vs. Regression (Backward Relevant Search) |
| 5. Goal Stack Planning & The Sussman Anomaly in Blocks World | 6. Planning Graphs (Graphplan) & 3 Classes of Mutual Exclusion (Mutex) Relations |
| 7. Reasoning Under Uncertainty: Probability Axioms & Conditional Independence | 8. Bayes' Rule & Full Joint Probability Distribution Formulations |
| 9. Bayesian Belief Networks (BBN): Directed Acyclic Graphs & CPT Matrices | 10. Direction-Dependent Separation (d-separation): Serial, Diverging & Collider Links |
| 11. Exact Inference (Variable Elimination) vs. Approximate Monte Carlo Sampling | 12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions) |

Topic 1 & 2: Classical Planning & The STRIPS Representation

Planning is the process of generating a sequence of actions that transforms an initial state S_0 into a desired goal state G .

The STRIPS (Stanford Research Institute Problem Solver) Language

- **State:** A conjunction of positive ground function-free literals (e.g., $\text{On}(A, B) \wedge \text{OnTable}(B) \wedge \text{Clear}(A) \wedge \text{HandEmpty}$). Closed-world assumption applies.
- **Goal:** A conjunction of positive literals that must hold true in the final state.
- **Action Schema** $\text{Action}(a)$: Formally specified by 3 components:
 1. **Preconditions**(a): Literals that must be satisfied before action a can be legally executed.
 2. **Add-List**(a): Literals that become True after executing action a .
 3. **Delete-List**(a): Literals that become False and are removed after executing action a .

STRIPS Operator Schema: Stacking Block A onto Block B

```

Action: Stack(x, y)
Preconditions: Clear(y) ^ Holding(x)
Delete-List:   Clear(y), Holding(x)
Add-List:      On(x, y), Clear(x), HandEmpty
  
```

Topic 5: Goal Stack Planning & The Sussman Anomaly

Goal Stack Planning uses a Last-In First-Out (LIFO) stack to hold goals and subgoals. It handles multiple goals sequentially by planning for one subgoal, which may inadvertently undo previously achieved subgoals:

The Sussman Anomaly (Interleaved Non-Linear Goals)

In the Blocks World, given Initial State: $\text{On}(C, A) \wedge \text{OnTable}(A) \wedge \text{OnTable}(B)$ and Goal State: $\text{On}(A, B) \wedge \text{On}(B, C)$.

- If the planner achieves $\text{On}(A, B)$ first, it puts A on B . But to put B on C , it must unstack A , undoing its first goal!
- If it achieves $\text{On}(B, C)$ first, C has A on top, requiring unstacking.
- *Resolution*: Requires **Partial-Order / Non-Linear Planning** that interleaves sub-plans rather than assuming linear independence of subgoals.

Topic 6: Planning Graphs & Mutual Exclusion (Mutex) Relations

A **Planning Graph** is a directed leveled graph consisting of alternating proposition levels ($P_0, P_1, \dots$) and action levels ($A_0, A_1, \dots$). Mutex links between actions indicate that two actions cannot occur simultaneously:

Mutex Class	Condition & Geometric Cause	Example in Blocks World
1. Inconsistent Effects	One action negates an effect added by the other action.	'Stack(A, B)' adds 'On(A, B)' while 'Unstack(A, B)' deletes 'On(A, B)'.
2. Interference	An effect of one action is the negation of a precondition of the other.	'Stack(A, B)' deletes 'Clear(B)', which is a precondition of 'Stack(C, B)'.
3. Competing Needs	A precondition of one action is mutually exclusive with a precondition of the other.	'Pickup(A)' requires 'HandEmpty', while 'Stack(B, C)' requires 'Holding(B)'.

Topic 7 & 8: Probabilistic Reasoning & Bayes' Rule

1. Conditional Probability & Product Rule:

$$P(A | B) = \frac{P(A \wedge B)}{P(B)} \implies P(A \wedge B) = P(A | B)P(B) = P(B | A)P(A)$$

2. Bayes' Rule Formula:

$$P(Y | X) = \frac{P(X | Y)P(Y)}{P(X)} = \frac{P(X | Y)P(Y)}{\sum_y P(X | y)P(y)}$$

Where $P(Y)$ is the *Prior Probability*, $P(X | Y)$ is the *Likelihood*, and $P(Y | X)$ is the *Posterior Probability*.

Topic 9 & 10: Bayesian Belief Networks (BBN) & d-separation

A **Bayesian Network** is a Directed Acyclic Graph (DAG) where nodes represent random variables and directed edges represent direct causal/probabilistic dependencies, annotated with Conditional Probability Tables (CPTs).

Full Joint Distribution Factorization Formula

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i | \text{Parents}(X_i))$$

Reduces the number of parameters needed to represent an n -variable boolean domain from $2^n - 1$ to $O(n \cdot 2^k)$, where k is the maximum number of parents per node!

The 3 Structural Connections of d-separation (Direction-Dependent Separation):

Connection Topology	Graph Structure & Dependency Status	Blocking Condition
1. Serial / Causal Chain	$X \rightarrow Z \rightarrow Y$	X and Y are conditionally independent given Z (Z is observed / instantiated).
2. Diverging / Common Cause (Fork)	$X \leftarrow Z \rightarrow Y$	X and Y are conditionally independent given Z (Z is observed).
3. Converging / Collider (V-Structure)	$X \rightarrow Z \leftarrow Y$	X and Y are independent when Z is unobserved . If Z or any of its descendants is observed, X and Y become dependent (Explaining Away effect).

 Top BIT Mesra Exam Questions & Answers (Module IV)

Q1. Given the classic Burglar Alarm Bayesian Network (Burglary, Earthquake, Alarm, JohnCalls, MaryCalls), compute the joint probability $P(B \wedge \neg E \wedge A \wedge J \wedge \neg M)$. (8 Marks)

Applying the Bayesian Network factorization chain rule:

$$P(B \wedge \neg E \wedge A \wedge J \wedge \neg M) = P(B) \cdot P(\neg E) \cdot P(A \mid B, \neg E) \cdot P(J \mid A) \cdot P(\neg M \mid A)$$

Substituting standard CPT values ($P(B) = 0.001, P(\neg E) = 0.998, P(A \mid B, \neg E) = 0.94, P(J \mid A) = 0.90, P(\neg M \mid A) = 0.30$):

$$= 0.001 \times 0.998 \times 0.94 \times 0.90 \times 0.30 = 0.000253 \text{ } (\approx 0.0253\%)$$

Q2. Explain the 3 mutex conditions in Graphplan with diagrams. (8 Marks)

- 1. **Inconsistent Effects:** Action 1 adds proposition p while Action 2 deletes p . They cannot occur concurrently without contradictory states.
- 2. **Interference:** Action 1 deletes proposition p which is required as a precondition by Action 2.
- 3. **Competing Needs:** A precondition of Action 1 and a precondition of Action 2 are mutually exclusive at the preceding proposition level.